Lesson notes · for you

Fast answers: indexes

Data engineering · Level 3 · Lesson 4

Time	Materials
16 min	Your phone with Darsi, for the Python steps; A friend, for “Guess my number”

Learning goals

- Explain why reading every row gets slow.
- Count the looks of a linear and a binary search in Python.
- Decide which columns to index.

Activity: Guess my number

1. Pairs (2 min): One partner secretly picks a number from 1 to 100.
2. Guess (5 min): First, guess 1, 2, 3… in order and count the guesses.
3. Halve (5 min): Now always guess the middle of what’s left, with “higher” or “lower” as the answer. Count again.
4. Compare (4 min): Compare the counts. Why does halving never need more than 7?

Let pupils keep score on paper; the record of guesses shows the pattern.

Discussion

- Why does a linear search get ten times slower on ten times more rows?
- Why does halving need the data to be sorted?
- What does an index cost?

Check yourself

- Write how many looks halving needs for 1,000 rows, and why.

Try it this week

Play “Guess my number” from 1 to 100 with a friend. Always guess the middle. Can you always win in seven guesses? Why seven?

Worksheet

Fast answers: indexes

Data engineering · Level 3 · Lesson 4

Name _____ Class _____ Date _____

1 **Why not put an index on every column?**

- ◯ SQL allows only one index
- ◯ Each index takes space and slows every new row
- ◯ Indexes make answers wrong

2 **Linear search. linear(ids, target) should return how many looks it took to find target (or to give up). For [5, 3, 9, 1] and 9, that's 3 looks. Fix the off-by-one mistake.**

```
def linear(ids, target):
    looks = 0
    for x in ids:
        if x == target:
            return looks
        looks = looks + 1
    return looks
```

Write your program:

3 **How many times can you halve a million before one is left? What does this program print?**

```
n = 1000000
looks = 0
while n > 1:
    n = n // 2
    looks = looks + 1
print(looks)
```

- ◯ 500000
- ◯ 1000
- ◯ 19

4 **Binary search on a sorted list. binary(ids, target) should halve the search each look and return how many looks it took. One comparison sends it the wrong way: find it and fix it. From 1 to 100, it should never need more than 7 looks.**

```
def binary(ids, target):
    lo, hi = 0, len(ids) - 1
    looks = 0
    while lo <= hi:
        mid = (lo + hi) // 2
        looks = looks + 1
        if ids[mid] == target:
            return looks
        elif ids[mid] > target:
            lo = mid + 1
        else:
            hi = mid - 1
    return looks
```

Write your program:

5 **A school library's loans table is searched by member and by book every day. Which columns are worth an index? Choose all that apply.**

- ☐ The colour of the book's cover
- ☐ member_id
- ☐ notes, which nobody searches
- ☐ book_id

✎ **Where in real life do you halve a search, and where do you read one by one?**

Check your answers

Fast answers: indexes

Data engineering · Level 3 · Lesson 4

1 **Each index takes space and slows every new row**

Indexes trade space and slower writes for faster searches: every new row must be added to every index.

2

```
def linear(ids, target):
    looks = 0
    for x in ids:
        looks = looks + 1
        if x == target:
            return looks
    return looks
```

Each item you check is one look, including the one you find, so count before comparing.

3 **19**

Halving a million again and again reaches 1 after 19 steps; with the final check, that's about 20 looks.

4

```
def binary(ids, target):
    lo, hi = 0, len(ids) - 1
    looks = 0
    while lo <= hi:
        mid = (lo + hi) // 2
        looks = looks + 1
        if ids[mid] == target:
            return looks
        elif ids[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return looks
```

lo and hi mark the part that's left. When the middle is smaller than the target, the target must be above it, so lo moves past mid; otherwise hi does.

5 **member_id; book_id**

Index what you search, join or sort by often.